Ibadullah 25K-6544

LAB 08 TASKS

Task 1:

```
#include <stdio.h>
int main() {
    int grades[4][3];
    int r, c;
    int tscore;
    float avg;
    for (r = 0; r < 4; r++) {
        printf("Input grades for student %d (3): ", r+1);
        for (c = 0; c < 3; c++) {
            scanf("%d", &grades[r][c]);
        }
    }
    printf("\n--- Student Total Scores ---\n");
    for (r = 0; r < 4; r++) {
        tscore = 0;
        for (c = 0; c < 3; c++) {
            tscore += grades[r][c];
        }
        printf("Total for student %d is %d\n", r + 1, tscore);
    }
    printf("\n--- Subject Averages ---\n");
    for (c = 0; c < 3; c++) {
        tscore = 0;
        for (r = 0; r < 4; r++) {
            tscore += grades[r][c];
        }
```

```
        avg = tscore / 4.0;
        printf("Average for subject %d is %.2f\n", c + 1, avg);
    }
    return 0;
}
```

```
Input grades for student 1 (3): 88
91
97
Input grades for student 2 (3): 66
69
74
Input grades for student 3 (3): 82
79
80
Input grades for student 4 (3): 98
95
94

--- Student Total Scores ---
Total for student 1 is 276
Total for student 2 is 209
Total for student 3 is 241
Total for student 4 is 287

--- Subject Averages ---
Average for subject 1 is 83.50
Average for subject 2 is 83.50
Average for subject 3 is 86.25
```

Task 2:

```
#include <stdio.h>
int main() {
    int cinema[5][6];
    int r, c;
    int emptySeats= 0;
    int highestBooked = -1;
    int busiestRow = 0;
    int rowBookings;
    for (r = 0; r < 5; r++) {
        printf("Input status for row %d (0 or 1): ", r+1);
        for (c = 0; c < 6; c++) {
            scanf("%d", &cinema[r][c]);
        }
```

```
    }
    for (r = 0; r < 5; r++) {
        rowBookings = 0;
        for (c = 0; c < 6; c++) {
            if (cinema[r][c] == 0) {
                emptySeats += 1;
            }
            if (cinema[r][c] == 1) {
                rowBookings += 1;
            }
        }
        if (rowBookings > highestBooked) {
            highestBooked = rowBookings;
            busiestRow = r + 1;
        }
}
    printf("\nAvailable seats left: %d\n", emptySeats);
    printf("The row with the most bookings is: Row %d\n", busiestRow);
    return 0;
}
```

```
Input status for row 1 (0 or 1): 0 1 0 1 1 0
Input status for row 2 (0 or 1): 1 0 0 1 0 1
Input status for row 3 (0 or 1): 1 1 1 1 0 1
Input status for row 4 (0 or 1): 0 0 1 1 1 0
Input status for row 5 (0 or 1): 1 1 0 1 0 1

Available seats left: 12
The row with the most bookings is: Row 3

--------------------------------
Process exited after 39.91 seconds with return value 0
Press any key to continue . . .
```

Task 3:

```
#include <stdio.h>

int main() {
```

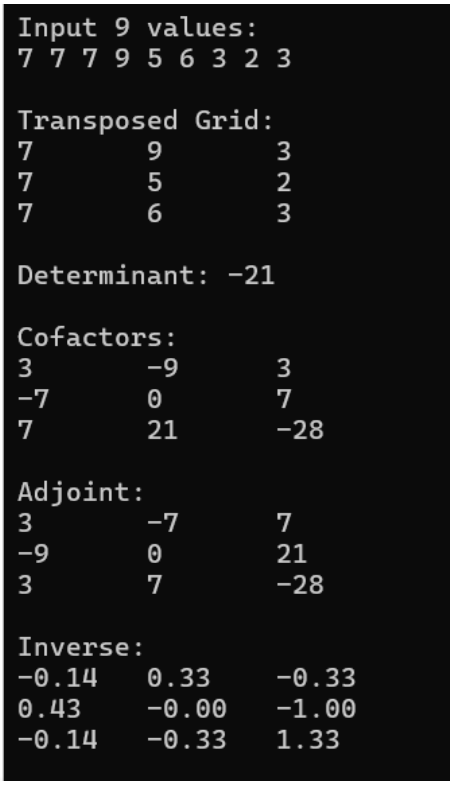
```
    float weather[7][3];
    int d, t;
    float highestTemp;
    float total,avg;
    for (d = 0; d < 7; d++) {
        printf("Input temperatures for day %d (morning afternoon night): ", d+1);
        for (t = 0; t < 3; t++) {
            scanf("%f", &weather[d][t]);
        }
    }
    highestTemp = weather[0][0];
    printf("\n--- Average Temp Per Day ---\n");
    for (d = 0; d < 7; d++) {
        total = 0;
        for (t = 0; t < 3; t++) {
            total += weather[d][t];

            if (weather[d][t] > highestTemp) {
                highestTemp = weather[d][t];
            }
        }
        avg = total / 3.0;
        printf("Average for day %d is %.2f\n", d + 1, avg);
    }
    printf("\nMax recorded temp this week: %.2f\n", highestTemp);
    return 0;
}
```

```
Input temperatures for day 1 (morning afternoon night): 22 30 25
Input temperatures for day 2 (morning afternoon night): 24 32 26
Input temperatures for day 3 (morning afternoon night): 23 31 24
Input temperatures for day 4 (morning afternoon night): 25 34 28
Input temperatures for day 5 (morning afternoon night): 26 35 29
Input temperatures for day 6 (morning afternoon night): 24 33 27
Input temperatures for day 7 (morning afternoon night): 21 28 23

--- Average Temp Per Day ---
Average for day 1 is 25.67
Average for day 2 is 27.33
Average for day 3 is 26.00
Average for day 4 is 29.00
Average for day 5 is 30.00
Average for day 6 is 28.00
Average for day 7 is 24.00

Max recorded temp this week: 35.00
```

Task 4:

```
#include <stdio.h>
int main() {
    int grid[3][3], adjoint[3][3];
    int row, col, x, y, d = 0;
    printf("Input 9 values:\n");
    for (row = 0; row < 3; row++)
        for (col = 0; col < 3; col++)
            scanf("%d", &grid[row][col]);
    printf("\nTransposed Grid:\n");
    for (row = 0; row < 3; row++) {
        for (col = 0; col < 3; col++) printf("%d\t", grid[col][row]);
        printf("\n");
    }
    d = grid[0][0]*(grid[1][1]*grid[2][2] - grid[1][2]*grid[2][1])
      - grid[0][1]*(grid[1][0]*grid[2][2] - grid[1][2]*grid[2][0])
      + grid[0][2]*(grid[1][0]*grid[2][1] - grid[1][1]*grid[2][0]);
    printf("\nDeterminant: %d\n\nCofactors:\n", d);
    for (row = 0; row < 3; row++) {
        for (col = 0; col < 3; col++) {
            int minor[4], p = 0;
            for (x = 0; x < 3; x++)
                for (y = 0; y < 3; y++)
```

```
                if (x != row && y != col) minor[p++] = grid[x][y];
            int calc = (minor[0]*minor[3]) - (minor[1]*minor[2]);
            if ((row + col) % 2 != 0) calc *= -1;
            adjoint[col][row] = calc;
            printf("%d\t", calc);
        }
        printf("\n");
    }
    printf("\nAdjoint:\n");
    for (row = 0; row < 3; row++) {
        for (col = 0; col < 3; col++) printf("%d\t", adjoint[row][col]);
        printf("\n");
    }
    if (d == 0) printf("\nNo Inverse.\n");
    else {
        printf("\nInverse:\n");
        for (row = 0; row < 3; row++) {
            for (col = 0; col < 3; col++) printf("%.2f\t", adjoint[row][col] / (float)d);
            printf("\n");
        }
    }
    return 0;
}
```



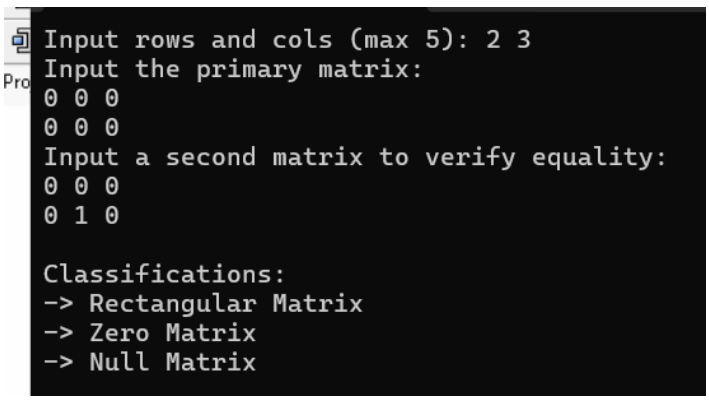
Task 5

```
#include <stdio.h>
int main() {
    int gridA[5][5], gridB[5][5];
    int rows, cols, x, y;
    int isZero = 1, isIdent = 1, isDiag = 1, isScl = 1;
    int isUp = 1, isLow = 1, isSym = 1, isSkew = 1, isEq = 1;
    int d = 0, checkD = 0, firstElement;

    printf("Input rows and cols (max 5): ");
    scanf("%d %d", &rows, &cols);

    printf("Input the primary matrix:\n");
    for (x = 0; x < rows; x++) {
        for (y = 0; y < cols; y++) {
            scanf("%d", &gridA[x][y]);
        }
    }
```

```
printf("Input a second matrix to verify equality:\n");
for (x = 0; x < rows; x++) {
    for (y = 0; y < cols; y++) {
        scanf("%d", &gridB[x][y]);
    }
}
firstElement = gridA[0][0];

for (x = 0; x < rows; x++) {
    for (y = 0; y < cols; y++) {
        if (gridA[x][y] != 0) isZero = 0;
        if (gridA[x][y] != gridB[x][y]) isEq = 0;

        if (rows == cols) {
            if (x == y) {
                if (gridA[x][y] != 1) isIdent = 0;
                if (gridA[x][y] != firstElement) isScl = 0;
                if (gridA[x][y] != 0) isSkew = 0;
            } else {
                if (gridA[x][y] != 0) {
                    isIdent = 0;
                    isDiag = 0;
                    isScl = 0;
                }
                if (x > y && gridA[x][y] != 0) isUp = 0;
                if (x < y && gridA[x][y] != 0) isLow = 0;
                if (gridA[x][y] != gridA[y][x]) isSym = 0;
                if (gridA[x][y] != -gridA[y][x]) isSkew = 0;
            }
        }
    }
```

```
}
printf("\nClassifications:\n");
if (rows == cols) printf("-> Square Matrix\n");
if (rows != cols) printf("-> Rectangular Matrix\n");
if (rows == 1) printf("-> Row Matrix\n");
if (cols == 1) printf("-> Column Matrix\n");

if (isZero == 1) {
    printf("-> Zero Matrix\n");
    printf("-> Null Matrix\n");
}
if (rows == cols) {
    if (isIdent == 1) printf("-> Identity Matrix\n");
    if (isDiag == 1) printf("-> Diagonal Matrix\n");
    if (isScl == 1) printf("-> Scalar Matrix\n");
    if (isUp == 1) printf("-> Upper Triangular Matrix\n");
    if (isLow == 1) printf("-> Lower Triangular Matrix\n");
    if (isSym == 1) printf("-> Symmetric Matrix\n");
    if (isSkew == 1) printf("-> Skew-Symmetric Matrix\n");

    if (rows == 1) {
        d = gridA[0][0];
        checkD = 1;
    } else if (rows == 2) {
        d = (gridA[0][0] * gridA[1][1]) - (gridA[0][1] * gridA[1][0]);
        checkD = 1;
    } else if (rows == 3) {
        d = gridA[0][0] * (gridA[1][1] * gridA[2][2] - gridA[1][2] * gridA[2][1]) -
            gridA[0][1] * (gridA[1][0] * gridA[2][2] - gridA[1][2] * gridA[2][0]) +
            gridA[0][2] * (gridA[1][0] * gridA[2][1] - gridA[1][1] * gridA[2][0]);
        checkD = 1;
```

```
        }
        if (checkD == 1) {
            if (d == 0) printf("-> Singular Matrix\n");
            else printf("-> Non-Singular Matrix\n");
        }
    }
    if (isEq == 1) printf("-> Equal Matrix\n");
    return 0;
}
```



Task 6

```
#include <stdio.h>
int main() {
    int a[3][3], b[3][3], ans[3][3];
    int rowsA, colsA, rowsB, colsB;
    int x, y, z;
    printf("input dimensions for Matrix 1(max 3 3): ");
    scanf("%d %d", &rowsA, &colsA);
    printf("input dimensions for Matrix 2(max 3 3): ");
    scanf("%d %d", &rowsB, &colsB);

    if (colsA != rowsB) {
        printf("\nError: Columns of the matrix 1 must match rows of the matrix 2.\n");
    } else {
        printf("\nInput values for first Matrix:\n");
```

```
for (x = 0; x < rowsA; x++) {
    for (y = 0; y < colsA; y++) {
        scanf("%d", &a[x][y]);
    }
}
printf("\nInput values for Second Matrix:\n");
for (x = 0; x < rowsB; x++) {
    for (y = 0; y < colsB; y++) {
        scanf("%d", &b[x][y]);
    }
}
for (x = 0; x < rowsA; x++) {
    for (y = 0; y < colsB; y++) {
        ans[x][y] = 0;
        for (z = 0; z < rowsB; z++) {
            ans[x][y] += a[x][z] * b[z][y];
        }
    }
}

printf("\nFirst Matrix\n");
for (x = 0; x < rowsA; x++) {
    for (y = 0; y < colsA; y++) {
        printf("%d\t", a[x][y]);
    }
    printf("\n");
}

printf("\nSecond Matrix\n");
for (x = 0; x < rowsB; x++) {
    for (y = 0; y < colsB; y++) {
```

```
            printf("%d\t", b[x][y]);
        }
        printf("\n");
    }

    printf("\nMultiplied Matrix\n");
    for (x = 0; x < rowsA; x++) {
        for (y = 0; y < colsB; y++) {
            printf("%d\t", ans[x][y]);
        }
        printf("\n");
    }
  }
  return 0;
}
```

```
input dimensions for Matrix 1(max 3 3): 3 2
input dimensions for Matrix 2(max 3 3): 2 3

Input values for first Matrix:
1 2
3 4
5 6

Input values for Second Matrix:
1 0 2
0 1 2

First Matrix
1       2
3       4
5       6

Second Matrix
1       0       2
0       1       2

Multiplied Matrix
1       2       6
3       4       14
5       6       22
```